Modern AI Foundations

A Complete Overview: From Classical Machine Learning to Large Language Models



A structured, code-first guide to the ideas behind modern artificial intelligence: the mathematics that powers it, the classical machine-learning engine at its heart, and the deep-learning revolution that brought us transformers and large language models — with curated sources for every step of the journey.

Prerequisites: Python (intermediate) • NumPy & Pandas (basic) • Calculus (basic)

Course Design Team

September 7, 2026

Contents

About this course 2

Prerequisites and required background 2

1 Introduction and the AI Landscape 4

1.1 Defining intelligence: weak vs. strong AI 4

1.2 From expert systems to data-driven AI 4

1.3 The modern stack: compute, cloud, and data pipelines 5

1.4 Ethics and safety: the non-optional module 6

2 Mathematics and Data Foundations 8

2.1 Linear algebra: the language of data 8

2.2 Probability and statistics: the language of uncertainty 9

2.3 Optimisation: how models learn 10

3 Classical Machine Learning: The Core Engine 12

3.1 Supervised learning 12

3.2 Unsupervised learning 13

3.3 Evaluation metrics: separating learning from memorising 14

4 Transition to Deep Learning and Modern AI 16

4.1 Neural networks: perceptrons, activations, backpropagation 16

4.2 Architectures: CNNs, RNNs, and transformers 17

4.3 Generative AI foundations: embeddings, latent space, and LLMs 18

Where to go next 21

Practical Coding Projects Series 22

Glossary 25

About this course

This document develops a four-module overview of modern artificial intelligence into a complete, self-contained study guide. It assumes no prior machine-learning experience, but it does assume the background summarised in the prerequisite matrix on the next page: comfortable **Python**, basic **NumPy & Pandas**, and **high-school calculus**. Everything else — from eigenvectors to attention — is built from scratch, in context, with citations to the primary literature and standard textbooks.

The four modules are deliberately sequential, as the roadmap below shows: the landscape and its ethics (*Module 1*), the mathematical foundations (*Module 2*), the classical machine-learning engine that still powers most industry deployments (*Module 3*), and the deep learning and generative-AI frontier (*Module 4*). All code snippets assume Python 3.9+ with `numpy`, `pandas`, `scikit-learn`, and (for Module 4) `transformers` installed.

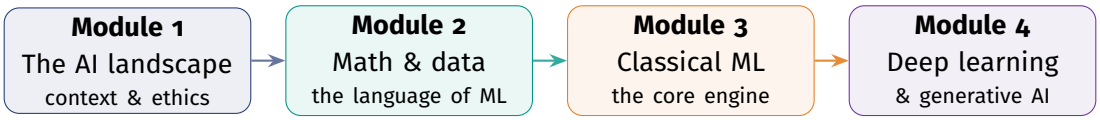


Figure 1. The learning path: each module builds directly on the one before it.

Throughout the guide you will meet the following recurring callout boxes:

Box	What it gives you
Module — at a glance	Focus, key concepts, and learning goals for each module.
Key concept	Definitions and intuitions worth memorising.
Hands-on with Python	Short, runnable code tied to the prerequisite skills.
Ethics & safety spotlight	Responsible-AI notes and open questions.
Check your understanding	A quick self-test before moving on.
Sources & further reading	Curated references backing each module.

Table 1. The callout boxes used throughout this guide.

Prerequisites and required background

The table below is the assumed starting point. None of the three rows requires expert mastery; the goal of each is narrow and practical.

Skill	Required level	Primary use case
Python programming	Intermediate	Implementing algorithms, data manipulation
NumPy & Pandas	Basic	Matrix operations and data cleaning
High-school calculus	Basic	Understanding how models learn (derivatives)

Table 2. Prerequisite matrix: the background assumed by this guide.

Python (intermediate). You should be comfortable writing functions, loops, list comprehensions, and simple classes, and know your way around a virtual environment. The course is deliberately

code-first: every mathematical idea is paired with a few lines of Python. A good refresher, free to read online, is **vanderplas2016**.

NumPy & Pandas (basic). You should be able to create arrays, slice them, and compute with them (`A @ B` for matrix products), and load a CSV into a DataFrame to clean and summarise it (**harris2020**; **mckinney2010**). The practice boxes assume exactly this level and no more.

Calculus (basic). You need the derivative as a slope, the power rule, and the chain rule. That is genuinely all — gradients, the engine of all learning in this course, are just derivatives collected into a vector.

The two calculus ideas you actually need

- **A derivative is a local slope:** $f'(x) \approx \dfrac{f(x+h) - f(x)}{h}$ — it tells you which direction is *downhill* and how steeply.
- **The chain rule composes slopes:** $\dfrac{d}{dx}\, f(g(x)) = f'(g(x))\, g'(x)$.

Backpropagation (Module 4) — the algorithm that trains every modern neural network — is nothing more than this second rule applied systematically, layer by layer.

Self-check before starting.

- Can you write a Python function that returns the mean and standard deviation of a list of numbers?
- Given two NumPy arrays, can you compute their dot product and the shape of `A @ B`?
- Can you differentiate $f(w) = w^2$ and say what $f'(3)$ means geometrically?

If any answer is “not yet”, spend a week with **vanderplas2016** and any standard calculus refresher; then return — everything else is built for you from here.

1 Introduction and the AI Landscape

Module 1 — at a glance

Focus. What artificial intelligence is (and is not), how the field evolved from hand-written rules to learning from data, and the infrastructure and ethical questions that shape modern practice.

Key concepts. Defining intelligence; weak vs. strong AI; expert systems vs. data-driven AI; GPUs, TPUs, cloud ecosystems and data pipelines; bias, alignment, copyright and environmental impact.

After this module you will be able to explain the difference between narrow and general AI, describe the paradigm shift from rules to data, sketch an end-to-end machine-learning pipeline, and name the main ethical questions attached to deploying an AI system.

1.1 Defining intelligence: weak vs. strong AI

Before we can build artificial intelligence we have to say what we mean by *intelligence*. **russell2021** organise the many definitions along two axes: are we concerned with *thought processes* or with *behaviour*, and should the benchmark be *human-like* performance or *rationality* — acting so as to achieve the best expected outcome? Alan Turing sidestepped the philosophy in 1950 with an operational test: if a machine's written answers are indistinguishable from a human's, is it not, for practical purposes, intelligent? (**turing1950**). Modern AI research mostly aims at rational behaviour on well-defined tasks — exactly what the learning algorithms of Modules 2–4 deliver.

Weak (narrow) AI vs. strong AI

Weak / narrow AI (today)	Strong AI / AGI (hypothetical)
Excels at one well-defined task: spam filters, image classifiers, next-token prediction	Reasons, plans and learns across arbitrary domains, as a human can
Learns from task-specific data; capability does not transfer	Would transfer knowledge flexibly to novel problems
Every deployed system, including today's LLMs, is narrow in this sense	Unknown whether or when it is achievable; an active research and policy debate

A second classical debate is worth knowing. Philosopher John Searle's *Chinese room* argument holds that symbol manipulation alone is not understanding: a person following rules could answer questions in Chinese without knowing any Chinese (**searle1980**). The argument has returned with force in the era of large language models, because fluent output is cheap while grounded understanding is hard to verify — a theme we revisit in the ethics spotlight below.

1.2 From expert systems to data-driven AI

The term *artificial intelligence* was coined at the 1956 Dartmouth workshop. For roughly the next three decades the dominant paradigm was *symbolic*: human knowledge was hand-encoded as if–then rules. Expert systems such as MYCIN encoded several hundred rules elicited from

physicians and impressed clinicians in blind evaluations — yet only inside their narrow domain. Rules are brittle, and extracting them from experts (the *knowledge-acquisition bottleneck*) is slow and expensive.

Machine learning inverts the pipeline: instead of programming rules, we collect examples of the desired behaviour and let an algorithm discover the rules. In **mitchell1997**'s classic formulation, a program *learns* from experience E with respect to task T and performance measure P if its performance at T, measured by P, improves with E. Deep learning pushes the inversion one step further by learning not only the rules but the *features* they operate on (**lecun2015**). Three results announced the new era: AlexNet cut the ImageNet top-5 error rate from roughly 26% to 16% by training a deep network on two consumer GPUs (**krizhevsky2012**); AlphaGo defeated the world Go champion using deep networks combined with search (**silver2016**); and transformer language models (Module 4) turned plain text prediction into a general-purpose technology.

The paradigm shift: from rules to data

Classical programming: rules + data $\longrightarrow$ answers
Machine learning: data + answers $\longrightarrow$ rules (the model)

Instead of hand-crafting the logic, we specify an objective and let an algorithm search for parameters that meet it — which is precisely why the mathematics of Modules 2 and 3 matters so much.

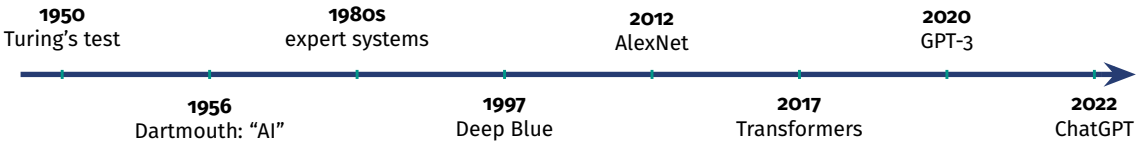


Figure 2. A selective history: from the founding of the field to the generative-AI era.

1.3 The modern stack: compute, cloud, and data pipelines

Compute: GPUs and TPUs. Training a modern model is, computationally, one thing: enormous numbers of matrix–vector products. A CPU optimises a handful of fast, sequential cores; a GPU packs thousands of simpler cores that perform these products in parallel. That is why **krizhevsky2012** could train a vision network that CPU-era attempts could not, and why hardware co-design now shapes chip roadmaps — Google's Tensor Processing Units, for instance, execute the tensor algebra of neural networks directly (**jouppi2017**). Memory bandwidth is the other half of the story: modern accelerators are as much about feeding data as about computing with it.

Cloud ecosystems. The same hardware is available to anyone, rented by the hour: a laptop, a browser, and a credit card are enough to requisition a cluster of GPUs with a reproducible software image. This democratised experimentation — and concentrated an enormous amount of AI capability in a handful of providers.

Data pipelines. Models live inside a loop, not a vacuum (Figure 3). Practitioners routinely spend the majority of project time collecting, cleaning, and validating data — which is exactly why Pandas is a prerequisite rather than an afterthought (**mckinney2010**).

Hands-on with Python

```python
import pandas as pd

df = pd.read_csv("orders.csv", parse_dates=["created"])
df = df.dropna(subset=["amount"])           # basic cleaning
df["revenue"] = df["amount"] * df["quantity"]
summary = (df.groupby("region")["revenue"]
.agg(["count", "mean", "sum"]))
print(summary)
```

Five lines of Pandas cover the ingest, clean, transform, aggregate loop that occupies most of a data scientist's day.

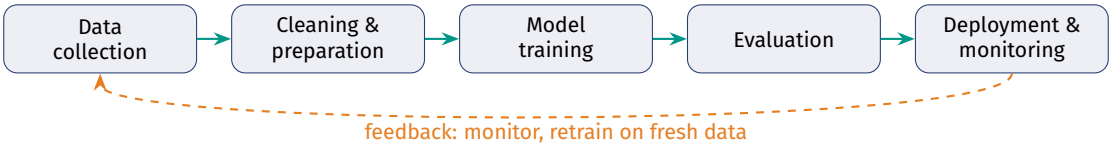


Figure 3. A modern data pipeline: the machine-learning lifecycle as a loop, not a line.

1.4 Ethics and safety: the non-optional module

Technical capability has outpaced governance. Four questions should accompany every system you build:

Four questions to ask of any AI system

Bias. Models inherit the patterns — including the unjust ones — in their training data. Commercial gender-classification systems were shown to misclassify darker-skinned women at far higher rates than lighter-skinned men (**buolamwini2018**). Audit data for representation gaps before, not after, deployment.

Alignment. Does the optimised objective match what humans actually want? Specification gaming and reward hacking are documented failure modes (**amodei2016**; **christian2020**); human-feedback training (Module 4) is one partial remedy.

Copyright. Models are trained on scraped text, images, and code. Provenance, licensing, and attribution of training data remain unsettled legal and ethical territory.

Environment. Training large models consumes substantial energy and emits carbon (**strubell2019**; **patterson2021**). Mitigations: efficient hardware, right-sized models, and transparency about training costs.

A fifth concern is specific to language models: fluency without grounding. **bender2021** famously warned that large language models are *stochastic parrots* — they reproduce statistical patterns of language, which is not the same as knowing anything. The practical consequence is cultural: verify outputs, cite sources, and treat generated text as a draft, never as evidence.

A practitioner's checklist.

- Define the task, the population served, and the failure modes *before* modelling.
- Audit training data for representation gaps and label quality.
- Choose metrics that price both error types (Module 3 shows how).

- Document limitations — model cards, datasheets, plain language.
- Plan for monitoring and human oversight after deployment.

Check your understanding

1. A chess engine, a spam filter, and a claimed “fully autonomous reasoning” startup demo: classify each as weak or strong AI, and justify the label.
2. Why did GPUs unlock deep learning roughly a decade before CPUs could? Answer in terms of the arithmetic that training requires.
3. Your model screens job applications. Name two concrete harms and one mitigation for each.

Sources & further reading

- **russell2021**
- **turing1950**
- **searle1980**
- **lecun2015**
- **krizhevsky2012**
- **silver2016**
- **jouppi2017**
- **buolamwini2018**
- **amodei2016**
- **christian2020**
- **strubell2019**
- **patterson2021**
- **bender2021**

2 Mathematics and Data Foundations

Module 2 — at a glance

Focus. The small mathematical toolkit that quietly powers everything else in the course: linear algebra to *represent* data, probability to *quantify* uncertainty, and optimisation to *learn* from mistakes.

Key concepts. Vectors, matrices, eigenvalues and dimensionality reduction; Bayes' theorem, distributions and hypothesis testing; loss and cost functions, gradients, and gradient descent.

After this module you will be able to describe any dataset as linear-algebra objects, reason correctly about conditional probabilities, and explain — and implement — gradient descent from scratch.

Mathematics here is a working tool, not an exam subject. Each concept is introduced the moment it earns its keep, and each is paired with NumPy code that uses it.

2.1 Linear algebra: the language of data

Machine-learning data lives in vectors, matrices, and tensors. A house becomes a vector of measurements $\mathbf{x} = [\text{area}, \text{rooms}, \text{age}]$; a dataset of m houses a matrix $X \in \mathbb{R}^{m \times n}$; a colour image a $H \times W \times 3$ tensor. The dot product measures alignment between vectors (its normalised version, cosine similarity, is the workhorse of recommendation systems and semantic search), and a matrix–vector product $W\mathbf{x}$ is a *change of representation*: it rotates, stretches, and mixes coordinates. In Module 4 this becomes literal — every neural-network layer *is* one such matrix.

Eigenvalues and dimensionality reduction. Eigenvectors are the directions a matrix leaves unchanged except for scaling:

$$A\mathbf{v} = \lambda \mathbf{v}. \tag{1}$$

The covariance matrix Σ of a dataset is symmetric, so its eigenvectors are mutually orthogonal, and the eigenvector with the largest eigenvalue points along the direction of greatest variance. Projecting the data onto the top eigenvectors is *principal component analysis* (PCA):

$$\mathbf{v}_1 = \arg\max_{\|\mathbf{v}\|=1} \mathbf{v}^\top \Sigma \, \mathbf{v}, \tag{2}$$

i.e. the first principal component is the top eigenvector of Σ. Dropping low-variance directions keeps most of the signal in far fewer coordinates — compression, de-noising, and two-dimensional plots of hundred-dimensional data (**strang2019**). NumPy makes all of it concrete, and its compiled vector operations are orders of magnitude faster than Python loops (**harris2020**).

Hands-on with Python

```python
import numpy as np

X = np.random.default_rng(0).normal(size=(500, 20))
Xc = X - X.mean(axis=0)              # centre the columns
C  = (Xc.T @ Xc) / (X.shape[0]-1)    # covariance matrix
evals, evecs = np.linalg.eigh(C)     # symmetric solver
pcs = evecs[:, ::-1][:, :2]          # top-2 principal components
X2  = Xc @ pcs                       # 20-D data projected to 2-D
print(X2.shape, evals[::-1][:2])     # (500, 2) and the top variances
```

2.2 Probability and statistics: the language of uncertainty

Real data is noisy and the future is unknown, so every prediction in this course is ultimately a probability statement. A handful of distributions covers most of the terrain: the *Bernoulli* for yes/no outcomes, the *categorical* for classes (and, we shall see, for tokens), and the *Gaussian* for measurement noise and natural variation:

$$p(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right). \tag{3}$$

The single most useful formula in the toolkit is Bayes' theorem:

$$P(A \mid B) \;=\; \frac{P(B \mid A)\,P(A)}{P(B)}. \tag{4}$$

Bayes' theorem, in words

Posterior $\propto$ likelihood $\times$ prior: what you believe *after* seeing evidence is proportional to how well each hypothesis would have produced that evidence, weighted by what you believed *before*.

A medical example shows why intuition fails without it. A test with 99% sensitivity and a 5% false-positive rate, applied to a disease with 1% prevalence, yields

$$P(\text{disease} \mid +) = \frac{P(+ \mid \text{disease})\,P(\text{disease})}{P(+)} \tag{5}$$

$$= \frac{0.99 \times 0.01}{0.99 \times 0.01 + 0.05 \times 0.99} \;\approx\; 0.17\,: \tag{6}$$

only one positive in six is a true case, because the disease is rare to begin with. The same base-rate reasoning governs fraud alerts, security scanners, and classifier performance on imbalanced classes.

Hypothesis testing disciplines the question "is this pattern real?". State a null hypothesis (no effect), compute a test statistic, and report a p-value: the probability of data at least this extreme *if the null were true*. Conventions like $p < 0.05$ are useful servants and terrible masters — multiple comparisons and selective reporting ("p-hacking") invalidate them silently. Prefer effect sizes and confidence intervals whenever possible (**wasserman2004**). Finally, the *central limit theorem*: averages of many independent draws become approximately Gaussian, with standard error $\sigma/\sqrt{n}$ — the reason bigger samples shrink error bars, and the intuition underlying both A/B testing and the scaling laws of Module 4.

Hands-on with Python

```python
import numpy as np

rng = np.random.default_rng(42)
# 10,000 experiments: average of 40 Exponential(1) draws each
means = rng.exponential(scale=1.0, size=(10_000, 40)).mean(axis=1)
print(means.mean(), means.std())   # ~1.0 and ~1/sqrt(40) = 0.158
```

The standard deviation of the means matches $\sigma/\sqrt{n}$: the central limit theorem, verified in three lines.

2.3 Optimisation: how models learn

A **loss function** scores the error of a prediction on a single example; the **cost function** (or empirical risk) averages it over the dataset. For regression, the default is mean squared error,

$$J(\boldsymbol{\theta}) = \frac{1}{2m}\sum_{i=1}^{m}\big(h_{\boldsymbol{\theta}}(\mathbf{x}_i) - y_i\big)^2. \tag{7}$$

The entire game of machine learning is then: choose a flexible family of functions, define a loss, and find the parameters that minimise it.

The **gradient** $\nabla_{\boldsymbol{\theta}} J$ collects all partial derivatives into a vector — the direction of steepest *ascent*. Learning therefore walks the other way:

$$\boldsymbol{\theta}_{t+1} \;=\; \boldsymbol{\theta}_t \;-\; \eta\,\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_t), \tag{8}$$

where η is the *learning rate*. Too small and training crawls; too large and it overshoots and diverges (Figure 4). In practice the gradient is estimated on small random *mini-batches* of data — stochastic gradient descent (SGD) — trading a little noise for a lot of speed.

Adam and adaptive optimisers. Standard SGD uses a fixed step size for all parameters. The **Adam** optimiser (**kingma2015**) adapts learning rates individually by maintaining exponential moving averages of the gradient $g_t = \nabla_{\boldsymbol{\theta}} J$ (first moment m_t) and its square (second moment v_t):

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \tag{9}$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \qquad \boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \frac{\eta}{\sqrt{\hat{v}_t}+\epsilon}\,\hat{m}_t, \tag{10}$$

where $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$. First-moment momentum accelerates along constant directions, while second-moment scaling dampens steps for features with frequently large gradients, making Adam the default choice for deep learning and Transformers.

Vanishing and exploding gradients. In deep architectures, backpropagating loss gradients across L layers involves repeated matrix multiplication:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(1)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(L)}} \prod_{\ell=2}^{L} W^{(\ell)\top} \operatorname{diag}\Big(\phi'\Big(\mathbf{z}^{(\ell)}\Big)\Big)\,. \tag{11}$$

If the eigenvalues of $W^{(\ell)}$ or activation slopes ϕ' are strictly less than 1, gradients decay exponentially to zero (*vanishing gradients*), leaving early layers untrained. If greater than 1, gradients blow up to infinity (*exploding gradients*). Linear models avoid this because they lack hidden depth; neural networks require architectural safeguards (residual connections, normalization, ReLU) to survive depth. Linear models have convex, bowl-shaped costs, so descent finds the

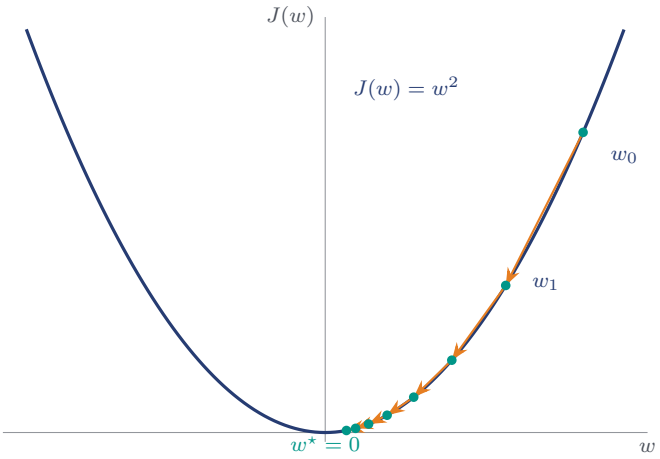


Figure 4. Gradient descent on $J(w) = w^2$ with learning rate $\eta = 0.15$: each step $w \leftarrow w - \eta\, 2w$ shrinks the distance to the minimum by 30%.

unique minimum; neural networks do not, yet SGD and Adam reliably find good solutions anyway — one of the field's pleasant empirical surprises.

Hands-on with Python

```python
w, lr = 5.0, 0.15
for step in range(8):
grad = 2 * w              # derivative of f(w) = w**2
w = w - lr * grad         # the update rule, eq. (8)
print(f"step {step}:  w = {w:.4f}")
# w -> 0.2882: convergence toward the minimum
```

Check your understanding

1. In plain words, what does an eigenvector of the covariance matrix tell you about a dataset?
2. In the medical-test example, what happens to $P(\text{disease} \mid +)$ as prevalence rises to 10%?
3. What goes wrong, mechanically, when the learning rate in equation (8) is set to 1.1 for $J(w) = w^2$?

Sources & further reading

- **strang2019**
- **wasserman2004**
- **boyd2004**
- **goodfellow2016**
- **bishop2006**
- **harris2020**
- **mckinney2010**
- **vanderplas2016**

3 Classical Machine Learning: The Core Engine

Module 3 — at a glance

Focus. The algorithms that still run the majority of real-world, structured-data workloads — and the evaluation discipline that separates real learning from memorisation.

Key concepts. Supervised learning: regression, decision trees, SVMs, ensembles. Unsupervised learning: K-Means clustering and anomaly detection. Evaluation: accuracy, precision, recall, F1, ROC-AUC.

After this module you will be able to choose an appropriate classical algorithm for a task, cluster unlabelled data, and — most importantly — evaluate any model honestly.

On the tabular, structured data of business (customers, transactions, sensors), classical machine learning frequently still beats glamorous deep models. It is also the best place to internalise the habits — splits, metrics, baselines — that govern everything later (**hastie2009**; **pedregosa2011**).

3.1 Supervised learning

Supervised learning starts from labelled pairs $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{m}$ and a split into *training*, *validation*, and *test* sets. If y is a number the task is *regression*; if y is a category it is *classification*.

Regression. The linear model $\hat{y} = \beta_0 + \boldsymbol{\beta}^{\top}\mathbf{x}$ has a closed-form least-squares solution,

$$\hat{\boldsymbol{\beta}} = (X^{\top}X)^{-1}X^{\top}\mathbf{y}, \tag{12}$$

in practice computed by QR factorisation or gradient descent for numerical stability. *Regularisation* fights overfitting by penalising large coefficients — ridge regression,

$$\hat{\boldsymbol{\beta}} = \arg\min_{\boldsymbol{\beta}} \ \|\mathbf{y} - X\boldsymbol{\beta}\|_2^2 + \alpha\|\boldsymbol{\beta}\|_2^2, \tag{13}$$

shrinks them smoothly; the lasso (ℓ_1) drives some exactly to zero, performing feature selection. *Logistic regression* turns a linear score into a class probability with the sigmoid function and is trained with the cross-entropy loss — the very same loss family that trains large language models in Module 4.

Decision trees and ensembles. A tree asks a sequence of yes/no questions ("area $>$ 120?"), choosing at each split the feature and threshold that most reduce impurity. Trees are fast, handle mixed data types, and are interpretable — and, alone, they overfit badly. *Random forests* average many de-correlated trees grown on random subsamples of rows and features (**breiman2001**); *gradient boosting* builds trees sequentially, each correcting the residual errors of the ensemble so far (**chen2016**). On tabular data these two are frequently the strongest off-the-shelf learners available.

Support vector machines. The SVM seeks the separating hyperplane with the *maximum margin* to the closest points, minimising the hinge loss

$$L = \max\bigl(0,\ 1 - y\,f(\mathbf{x})\bigr), \qquad y \in \{-1, +1\}, \tag{14}$$

and the *kernel trick* lets it build non-linear boundaries implicitly in high-dimensional feature space (**cortes1995**).

Feature scaling requirement. Algorithms that calculate spatial distances (K-Means, SVM) or compute gradient steps (Logistic/Linear Regression, Neural Networks) are highly sensitive to

Table 3. Choosing a first algorithm for tabular data.

Algorithm	Reach for it when	Strengths	Watch out
Linear / logistic regression	Baselines; small, clean data	Fast, interpretable, statistically principled	Underfits complex nonlinear structure
Decision tree	Interpretability is the product	Human-readable rules; no feature scaling	Single trees overfit badly
Random forest	Default robust choice	Variance reduction via bagging; few knobs	Less interpretable; larger memory
Gradient boosting (XGBoost)	Maximum accuracy on tables	State of the art on structured data	Needs tuning; can overfit small data
SVM	Small–medium data, many features	Max-margin generalisation; kernels	Scales poorly to very large datasets

feature scales: features with large magnitudes (e.g., house area in sq ft) dominate Euclidean metrics over features with small ranges (e.g., room count). Standardisation ($z = (x - \mu)/\sigma$ via `StandardScaler`) or min-max scaling is mandatory prior to training. By contrast, decision trees and tree ensembles are *scale-invariant*, as splits evaluate individual feature thresholds independently.

3.2 Unsupervised learning

Unsupervised learning finds structure without labels. The canonical clustering algorithm, **K-Means** (**macqueen1967**), partitions m points into K clusters by minimising within-cluster squared distance,

$$J = \sum_{k=1}^{K} \sum_{\mathbf{x}_i \in C_k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2, \tag{15}$$

by alternating two steps: (1) assign each point to its nearest centroid $\boldsymbol{\mu}_k$; (2) move each centroid to the mean of its members. It converges quickly, but only to a *local* optimum — hence multiple random restarts — and it implicitly assumes roundish, equally sized clusters. Choosing K is a design decision; the "elbow" of the cost curve offers a visual heuristic.

Quantitative cluster evaluation: Silhouette Score. To evaluate clustering quality without relying solely on visual heuristics, the **Silhouette Coefficient** (**rousseeuw1987**) measures how tightly grouped points are within their cluster compared to neighboring clusters:

$$s(i) = \frac{b(i) - a(i)}{\max\big(a(i),\, b(i)\big)}, \qquad s(i) \in [-1, +1], \tag{16}$$

where $a(i)$ is the mean distance between point i and all other points in the same cluster, and $b(i)$ is the mean distance to points in the nearest neighboring cluster. A mean score near $+1$ indicates well-separated, dense clusters; scores near 0 indicate overlapping clusters. Typical uses: customer segmentation, topic grouping, preprocessing.

Anomaly detection flags rare, suspicious items — fraud, failed sensors, rare medical readings. Statistical methods flag points in the tails of fitted distributions; distance- and density-based methods flag points far from any crowd; and the *isolation forest* exploits the fact that anomalies are easy to separate from everything else with few random splits (**liu2008**). Mixture models (**bishop2006**) give a probabilistic route, and autoencoders (Module 4) extend the idea to complex data.

3.3 Evaluation metrics: separating learning from memorising

A model that cannot be measured cannot be trusted. All classification metrics descend from the *confusion matrix*:

Table 4. The confusion matrix for binary classification.

	Predicted positive	**Predicted negative**
Actual positive	TP (hit)	FN (miss)
Actual negative	FP (false alarm)	TN (correct rejection)

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \qquad \text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}, \tag{17}$$

$$F_1 = 2\,\frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{18}$$

The metric is a *value judgement* about which errors matter. If 1% of transactions are fraudulent, a model that always says "not fraud" is 99% accurate and perfectly useless; a fraud system should trade precision for recall, while a spam filter should do the opposite. The **ROC curve** traces true- vs. false-positive rate as the decision threshold varies, and its area (AUC) — the probability that a random positive outranks a random negative — summarises threshold-free quality (**fawcett2006**).

Two further disciplines: **cross-validation** rotates the train/validation split (k-fold) to use data efficiently and to *measure variance* across folds; and the **bias–variance trade-off** names the two failure modes — underfitting (too simple, high bias) and overfitting (memorising the training set, high variance) — with regularisation, more data, or better inductive bias as the respective remedies (**hastie2009**).

Hands-on with Python

```
                    from sklearn.model_selection import train_test_split,
cross_val_score
                    from sklearn.ensemble import RandomForestClassifier
                    from sklearn.metrics import classification_report

                    X_tr, X_te, y_tr, y_te = train_test_split(
                    X, y, test_size=0.2, stratify=y, random_state=0)

                    clf = RandomForestClassifier(n_estimators=300, random_state=0)
                    print(cross_val_score(clf, X_tr, y_tr, cv=5))   # per-fold scores
                    clf.fit(X_tr, y_tr)
                    print(classification_report(y_te, clf.predict(X_te)))
```

Check your understanding

1. For a cancer-screening tool, which metric would you prioritise, and what is the cost of that choice?
2. Why does K-Means struggle with two concentric rings of points, and what does that reveal about its assumptions?
3. Your cross-validation scores vary from 0.62 to 0.91 across folds. What does that variance

tell you, and what would you do next?

Sources & further reading

- **mitchell1997**
- **hastie2009**
- **bishop2006**
- **cortes1995**
- **breiman2001**
- **chen2016**
- **macqueen1967**
- **liu2008**
- **fawcett2006**
- **pedregosa2011**

4 Transition to Deep Learning and Modern AI

Module 4 — at a glance

Focus. The leap from engineered features to learned representations: neural networks and backpropagation, the three architectures that shaped the field, and the mechanics of generative AI and large language models.

Key concepts. Perceptrons, activation functions, backpropagation; CNNs, RNNs, and transformers; embeddings, latent space, and next-token prediction.

After this module you will be able to explain how a neural network is trained, why transformers displaced recurrence, and what exactly a large language model is doing when it answers you.

The jump from classical ML to deep learning is a jump in *who designs the features*. A classical pipeline is given hand-crafted inputs; a deep network learns a hierarchy of them — edges, then textures, then parts, then objects (**lecun2015**). What engineering achieved by hand, depth achieves with data and compute.

4.1 Neural networks: perceptrons, activations, backpropagation

The **perceptron** (**rosenblatt1958**) is a single neuron: a weighted sum passed through a threshold,

$$\hat{y} = \mathbb{1}\{\mathbf{w}^\top \mathbf{x} + b > 0\} \in \{0, 1\} \quad (\text{or } \operatorname{sgn}(\mathbf{w}^\top \mathbf{x} + b) \in \{-1, +1\}). \tag{19}$$

It can learn any function that a straight line can separate — and provably cannot learn XOR, a limitation that chilled the field's first winter (**minsky1969**). The repair is to *stack* units and interleave **nonlinear activation functions** between layers:

$$\mathbf{a}^{(\ell)} = \phi\left(W^{(\ell)}\, \mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}\right), \qquad \ell = 1, \dots, L. \tag{20}$$

Without ϕ, the stack would collapse into a single linear map. The rectified linear unit, $\phi(z) = \max(0, z)$, became the default for being cheap, non-saturating, and sparse in its activations (**nair2010**). Such networks are universal approximators (**hornik1989**) — a statement about *capacity*, with no guarantee that training finds good parameters.

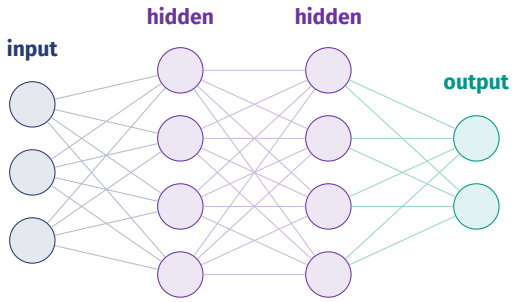


Figure 5. A fully connected 3–4–4–2 network. Every edge carries a learnable weight; each layer applies its matrix, then a nonlinearity.

Backpropagation (**rumelhart1986**) computes the gradient of the loss $\mathcal{L}$ with respect to every weight (the chain rule from Module 1) organised backwards through the layers. With $\delta^{(\ell)} \equiv$

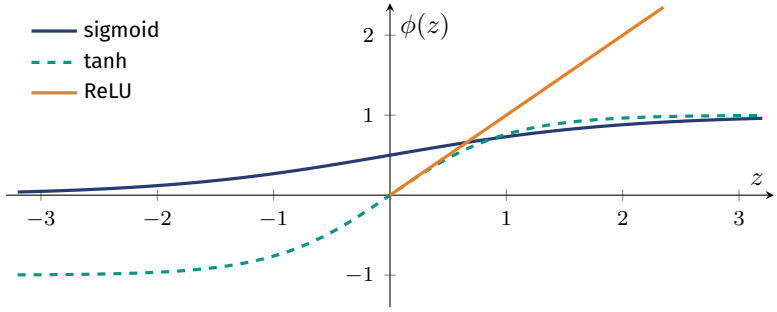


Figure 6. Common activation functions. ReLU's lack of saturation for $z > 0$ helped make deep networks trainable at scale (**nair2010**).

$\partial \mathcal{L}/\partial \mathbf{z}^{(\ell)}$:

$$\boldsymbol{\delta}^{(L)} = \nabla_{\mathbf{a}^{(L)}} \mathcal{L} \odot \phi'\big(\mathbf{z}^{(L)}\big), \tag{21}$$

$$\boldsymbol{\delta}^{(\ell)} = \big(W^{(\ell+1)\top} \boldsymbol{\delta}^{(\ell+1)}\big) \odot \phi'\big(\mathbf{z}^{(\ell)}\big), \qquad \ell = L-1, \dots, 1, \tag{22}$$

$$\frac{\partial \mathcal{L}}{\partial W^{(\ell)}} = \boldsymbol{\delta}^{(\ell)} \, \mathbf{a}^{(\ell-1)\top}, \tag{23}$$

where $\odot$ is element-wise multiplication. Backprop is a gradient-*computation* algorithm; the learning itself is still gradient descent (equation (8)), now run on mini-batches through frameworks such as PyTorch or JAX.

4.2 Architectures: CNNs, RNNs, and transformers

Convolutional networks (CNNs) exploit the structure of images: local patterns, and translation invariance. A convolution slides a small learnable filter K over the input,

$$(I * K)(i,j) = \sum_{m} \sum_{n} I(i-m,\, j-n)\, K(m,n), \tag{24}$$

so the *same* few weights detect the same pattern everywhere — drastically fewer parameters than a dense layer, plus built-in equivariance. Pooling layers shrink resolution. LeNet established the recipe for document recognition (**lecun1998**); AlexNet's GPU-trained depth ignited the modern era (**krizhevsky2012**).

Recurrent networks (RNNs) handle sequences by carrying a hidden state from step to step:

$$\mathbf{h}_t = \tanh\big(W_h \, \mathbf{h}_{t-1} + W_x \, \mathbf{x}_t + \mathbf{b}\big). \tag{25}$$

Training them across long sequences suffers from vanishing gradients; the **LSTM**'s gated memory cells were the decisive fix (**hochreiter1997**), dominating language, speech, and time series for two decades.

Transformers replaced recurrence with *attention* (**vaswani2017**). Each token emits a query, a key, and a value; the output for each position is a weighted average of all values, with weights given by query–key compatibility:

$$\mathrm{Attention}(Q,K,V) = \mathrm{softmax}\!\left(\frac{Q\,K^{\top}}{\sqrt{d_k}}\right) V. \tag{26}$$

Because every token can attend to every other in a single parallel step, transformers train vastly faster than RNNs on modern hardware — the property, more than any single equation, that made the modern era possible.

Positional encodings. Plain self-attention is permutation-invariant: shuffling input tokens yields identical attention outputs (permuted correspondingly). To inject sequence order, **positional encodings** are added to input embeddings. The original Transformer used fixed sinusoidal frequencies (**vaswani2017**):

$$PE_{(pos,2i)} = \sin\Big(\frac{pos}{10000^{2i/d_model}}\Big), \qquad PE_{(pos,2i+1)} = \cos\Big(\frac{pos}{10000^{2i/d_model}}\Big). \tag{27}$$

Modern LLMs predominantly use **Rotary Position Embeddings (RoPE)** (**su2024**), which rotate query and key vectors in complex 2D planes, incorporating relative distance directly into the dot product $QK^\top$.

Residual connections and normalization. Deep Transformer stacks survive vanishing gradients by wrapping every sub-layer with a **residual (skip) connection** (**he2016**) and **Layer Normalization**:

$$\mathbf{y} = \mathrm{LayerNorm}\big(\mathbf{x} + \mathcal{F}(\mathbf{x})\big). \tag{28}$$

Skip connections provide an unattenuated identity gradient highway ($\partial\mathbf{y}/\partial\mathbf{x} = I + \partial\mathcal{F}/\partial\mathbf{x}$), enabling gradient flow across hundreds of layers. Within five years the architecture had absorbed language, vision, audio, and protein-structure modelling.

Table 5. Three architectures at a glance.

Architecture	Designed for	Key idea	Landmark
CNN	Images, video, grids	Local filters plus weight sharing yield equivariant feature hierarchies	(**lecun1998**; **krizhevsky2012**)
RNN / LSTM	Sequences, time series	Recurrence carries state; gating fights vanishing gradients	(**hochreiter1997**)
Transformer	Text — and increasingly everything	Self-attention lets every token see every other; trains in parallel	(**vaswani2017**)

4.3 Generative AI foundations: embeddings, latent space, and LLMs

Embeddings. Word2vec showed that a shallow network trained on "predict the neighbours of a word" learns vectors with striking arithmetic structure: $\mathbf{w}_{\text{king}} - \mathbf{w}_{\text{man}} + \mathbf{w}_{\text{woman}} \approx \mathbf{w}_{\text{queen}}$ (**mikolov2013**). Meaning becomes geometry: similar things cluster, relations are (roughly) directions. The same trick now embeds users, products, images, and code.

Latent space. Generative models compress data into a lower-dimensional *latent* code from which they can reconstruct or create. Variational autoencoders learn a probabilistic latent space and sample from it (**kingma2014**); GANs pit a generator against a discriminator until the fakes become indistinguishable (**goodfellow2014**). Generation, in all cases, is decoding from a point in latent space.

Large language models. An LLM does one thing: predict the next *token* (a sub-word unit) given the previous ones. The network outputs a score z_j for every vocabulary entry, and the softmax turns scores into a distribution,

$$p(y = j \mid \mathbf{x}) = \frac{\exp(z_j)}{\sum_{k=1}^{V}\exp(z_k)}, \qquad \mathcal{L}_{\text{CE}} = -\log p(y_{\text{true}} \mid \mathbf{x}), \tag{29}$$

the cross-entropy loss — logistic regression's loss, at vocabulary scale. **Causal masking and LLM taxonomy.** For generative language modelling, transformers must not look into the future. A

causal mask M sets upper-triangular attention logits to $-\infty$:

$$M_{ij} = \begin{cases} 0 & \text{if } j \le i \\ -\infty & \text{if } j > i \end{cases}, \qquad \mathrm{CausalAttention}(Q,K,V) = \mathrm{softmax}\left(\frac{QK^\top + M}{\sqrt{d_k}}\right)V, \tag{30}$$

ensuring position i attends only to tokens $j \le i$. This establishes the three major Transformer design families:

Table 6. The three Transformer architecture families.

Family	Attention Type	Primary Use Case	Examples
Decoder-only	Causal / Masked	Autoregressive text generation	GPT-4, Llama 3, Claude
Encoder-only	Bidirectional	Representation, classification, search	BERT, RoBERTa
Encoder-Decoder	Cross + Causal	Translation, summarization	T5, BART

Pretraining minimises cross-entropy over trillions of tokens of web text; *scaling laws* showed that loss falls predictably as parameters, data, and compute grow (**kaplan2020**); GPT-3 demonstrated that sufficiently large models perform new tasks from a handful of in-context examples, with no weight updates (**brown2020**). Alignment then shapes behaviour: supervised instruction tuning, followed by reinforcement learning from human feedback, trains models to follow instructions and avoid harmful outputs (**christiano2017**; **ouyang2022**). At generation time a *temperature* knob interpolates between greedy and adventurous sampling.

How an LLM writes one token

1. Tokenise the prompt into sub-word tokens.
2. Embed each token as a vector (plus positional information).
3. Pass the sequence through stacked transformer blocks: each token attends to all previous tokens.
4. Map the hidden state vector of the final sequence position through a linear layer and a softmax over the vocabulary.
5. Sample the next token from that distribution.
6. Append it to the sequence and repeat — one full forward pass per generated token.

Big models, big responsibilities

The environmental and bias costs of Module 1 scale with model size (**strubell2019**; **patterson2021**; **bender2021**). Practical habits for the LLM era: verify claims against primary sources; prefer the smallest model that does the job; log and audit failures; disclose when output is machine-generated; and remember that fluency is evidence of statistical pattern, not of truth.

Hands-on with Python

```python
from transformers import pipeline

clf = pipeline("sentiment-analysis",
model="distilbert-base-uncased-finetuned-sst-2-english")
print(clf("This course is fantastic!"))
# [{'label': 'POSITIVE', 'score': 0.9998...}]
```

A fine-tuned transformer classifier in four lines — the first run downloads the model (**huggingface**).

Check your understanding

1. Remove the nonlinearity from an MLP: what functions can the resulting network represent, and why?
2. Give two concrete reasons transformers train faster than RNNs on the same hardware.
3. In the next-token formulation, what does "temperature = 0" correspond to?

Sources & further reading

- **rosenblatt1958**
- **minsky1969**
- **rumelhart1986**
- **hornik1989**
- **nair2010**
- **lecun1998**
- **hochreiter1997**
- **vaswani2017**
- **mikolov2013**
- **kingma2014**
- **goodfellow2014**
- **brown2020**
- **kaplan2020**
- **ouyang2022**
- **goodfellow2016**
- **bishop2024**

Where to go next

The guide ends; the field does not. A few well-marked paths, matched to what you now know:

- **Strengthen the mathematics:** (**strang2019**; **wasserman2004**; **boyd2004**).
- **Go deep on classical ML:** (**hastie2009**; **bishop2006**), alongside the scikit-learn documentation (**pedregosa2011**).
- **Deep learning proper:** (**goodfellow2016**; **bishop2024**), and the runnable-notebook textbook of **zhang2023**.
- **Practice-first:** the free online course of **fastai**.
- **Build with LLMs:** the Hugging Face ecosystem (**huggingface**), read alongside (**vaswani2017**; **brown2020**; **ouyang2022**).
- **Stay grounded:** (**christian2020**; **bender2021**).

Suggested mini-projects, one per module transition:

1. Predict housing prices from a public dataset (Modules 2–3: regression, regularisation, honest evaluation).
2. Cluster and anomaly-detect a transactions dataset (Module 3: K-Means, isolation forest, and metric trade-offs).
3. Fine-tune a small transformer classifier on your own texts (Module 4: embeddings, cross-entropy, transfer learning).

AI is not a spectator sport. The mathematics of Module 2 becomes real the first time a gradient-descent loop you wrote yourself converges; the abstractions of Module 4 become real the first time a model you fine-tuned answers a question in your own domain. Start small, cite your sources, question your metrics — and enjoy the ride.

Practical Coding Projects Series

To accompany the theoretical foundation developed across the four modules, a complete series of six production-grade coding projects is provided in the repository's `projects/` directory. Each project bridges theoretical concepts to runnable Python implementation using industry-standard frameworks (`NumPy`, `Pandas`, `Scikit-Learn`, `XGBoost`, `PyTorch`, and `Transformers`), with explicit emphasis on data cleaning, exploratory visualization, model diagnostics, resumable training, and inference pipelines.

Project 1: Data Pipeline, Exploratory Analysis & Algorithmic Bias Audit

Curriculum Mapping: Module 1 (The AI Landscape, Compute/Cloud, Data Pipelines, Ethics & Safety).

Overview & Learning Objectives: This project implements the end-to-end data engineering and ethical auditing pipeline that precedes model training. Students work with real-world clinical data (UCI Breast Cancer Diagnostic dataset) with injected corruptions and demographic proxy groups.

1. **Data Cleaning & Engineering:** Ingest messy tabular data, compute imputation statistics, and handle outliers using interquartile range (IQR) clipping.
2. **Exploratory Data Analysis (EDA):** Build publication-quality data quality visualizations (missingness profiles, KDE distributions, and correlation matrices).
3. **Ethical & Algorithmic Bias Auditing:** Audit demographic representation gaps, evaluate Disparate Impact Ratios (80% rule), and compute Statistical Parity and Equalized Odds metrics.

Execution Command:

```
python projects/01_data_analysis_and_ethics/data_pipeline_demo.py
```

Hands-on Exercises: (1) Extend `cleaning.py` to support dynamic one-hot encoding with high-cardinality handling; (2) Modify `bias_audit.py` to compute Equal Opportunity Difference ($\text{TPR}_{\text{unpriv}} - \text{TPR}_{\text{priv}}$); (3) Document demographic representation gaps in a formal model card.

Project 2: Math Engine from Scratch (Linear Algebra, Stats & Optimization)

Curriculum Mapping: Module 2 (Mathematics and Data Foundations).

Overview & Key Topics: Translates core mathematical mechanics into pure NumPy implementations from first principles.

1. **Linear Algebra & Dimensionality Reduction:** `PCAFromScratch` via eigen-decomposition of the covariance matrix ($A\mathbf{v} = \lambda\mathbf{v}$) and Singular Value Decomposition ($X = U\Sigma V^\top$).
2. **First-Order Optimization Algorithms:** Pure NumPy implementation of Vanilla Gradient Descent, Momentum, RMSprop, and Adam optimizers with trajectory tracking.
3. **Probability & Statistics:** Central Limit Theorem verification ($\sigma_{\bar{x}} = \sigma/\sqrt{n}$) and exact Bayesian Beta-Binomial conjugate updating.

Execution Command:

```
python projects/02_math_and_optimization/run_math_sandbox.py
```

Hands-on Exercises: (1) Implement Nesterov Accelerated Gradient (NAG) in `optimizers.py`; (2) Implement automatic component selection for $\geq 95\%$ variance retention in `pca_from_scratch.py`; (3) Simulate vanishing and exploding gradients across a 10-layer linear transformation stack.

Project 3: Production-Ready Classical Machine Learning Pipeline & Diagnostics

Curriculum Mapping: Module 3 (Classical Machine Learning: The Core Engine).

Overview & Key Capabilities: Constructs an end-to-end machine learning pipeline covering supervised classification, unsupervised clustering, anomaly detection, and diagnostic learning curves.

1. **Pipeline Preprocessing:** Scikit-Learn `ColumnTransformer` handling numerical scaling and categorical encoding.
2. **Model Evaluation & Diagnostics:** Dynamic Learning Curves plotting train vs. validation ROC-AUC scores over dataset subsets to diagnose High Variance (overfitting) vs. High Bias (underfitting).
3. **Unsupervised Mechanics:** K-Means clustering evaluated with Silhouette Coefficients ($s(i) = \frac{b(i)-a(i)}{\max(a(i),b(i))}$) across candidate K, and Isolation Forest anomaly scoring.
4. **Production Serving:** Pipeline serialization (`joblib`) and low-latency `ClassicalInferenceService`.

Execution Command:

```
python projects/03_classical_machine_learning/train_pipeline.py
```

Hands-on Exercises: (1) Add `RandomizedSearchCV` hyperparameter tuning over XGBoost trees; (2) Implement a Validation Curve for regularization strength C in Logistic Regression; (3) Compare SMOTE vs. `class_weight='balanced'` on imbalanced data.

Project 4: Deep Learning Foundations & Resumable Training Pipeline

Curriculum Mapping: Module 4 (Neural Networks, Backpropagation, PyTorch/Keras foundations).

Overview & Key Capabilities: Bridges manual matrix calculus to deep learning framework primitives and enterprise checkpointing.

1. **NumPy Backprop Engine:** 2-layer Neural Network with manual matrix calculus derivatives ($\boldsymbol{\delta}^{(L)} = \nabla_{\mathbf{a}^{(L)}} \mathcal{L} \odot \phi'(\mathbf{z}^{(L)})$, $\boldsymbol{\delta}^{(\ell)} = (W^{(\ell+1)\top} \boldsymbol{\delta}^{(\ell+1)}) \odot \phi'(\mathbf{z}^{(\ell)})$).
2. **PyTorch vs. Keras Parity:** Modular `PyTorchMLP` implementation side-by-side with Keras functional equivalent code.
3. **Resumable Training Architecture:** `CheckpointManager` saving complete PyTorch state dictionaries (`model_state_dict`, `optimizer_state_dict`, `scheduler_state_dict`, epoch, and metric history) enabling seamless pause/resume capabilities across process interruptions.
4. **Inference Service:** High-throughput PyTorch batch inference pipeline.

Execution Command:

```
python projects/04_deep_learning_foundations/train_resumable.py
```

Hands-on Exercises: (1) Extend `mlp_from_scratch.py` to support arbitrary L-layer deep stacks with Softmax output; (2) Add Early Stopping patience counter state persistence to `checkpoint_manager.py`; (3) Evaluate `CosineAnnealingLR` scheduling.

Project 5: Computer Vision & Sequential Architectures (CNNs & LSTMs)

Curriculum Mapping: Module 4 (Architectures: CNNs and RNNs/LSTMs).

Overview & Key Capabilities: Implements the two foundational deep learning architectures for spatial grid data and temporal sequences.

1. **Convolutional Neural Networks (CNNs):** PyTorch 2D CNN with Residual Skip Connections ($\mathbf{y} = \text{LayerNorm}(\mathbf{x} + \mathcal{F}(\mathbf{x}))$), BatchNorm, and spatial pooling.
2. **Recurrent Networks (LSTMs):** Multi-layer PyTorch LSTM sequence classifier ($\mathbf{h}_t = \tanh(W_h \mathbf{h}_{t-1} + W_x \mathbf{x}_t + \mathbf{b})$) with memory gating.
3. **Trainer Engine:** `DeepModelTrainer` with `CosineAnnealingLR` scheduling and checkpointing.

Execution Command:

```
python projects/05_vision_and_sequence_models/train_vision_sequence.py
```

Hands-on Exercises: (1) Extend `cnn_model.py` to plot activation feature maps of intermediate convolutional layers; (2) Implement a Bidirectional LSTM in `lstm_model.py` and compare classification metrics.

Project 6: Transformer Architectures & LLM Fine-Tuning Pipeline

Curriculum Mapping: Module 4 (Transformers, Generative AI & Large Language Models).

Overview & Key Mechanics: Implements self-attention from scratch, fine-tunes pretrained models, and controls next-token decoding.

1. **Attention from Scratch:** Scaled Dot-Product Attention ($\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top + M}{\sqrt{d_k}}\right)V$) and Causal Attention Masking ($M_{ij} = -\infty$ for $j > i$).
2. **Transformer Block:** Multi-Head Attention, Pre-Layer Normalization, GELU Feed-Forward Network, and Residual Skip Connections.
3. **Hugging Face Fine-Tuning:** Pretrained transformer fine-tuning pipeline (`DistilBERT`/`BERT`).
4. **LLM Generation Control:** Next-token sampling engine supporting Temperature scaling (T), Top-k truncation, and Top-p (nucleus) sampling.

Execution Command:

```
python projects/06_transformers_and_llms/train_transformer.py
```

Hands-on Exercises: (1) Extend `transformer_scratch.py` to implement Rotary Position Embeddings (RoPE); (2) Implement Repetition Penalty (penalty ≥ 1.0) in `llm_inference.py` to reduce duplicate generation.

Glossary

Table 7. Quick reference for the vocabulary used in this guide.

Term	Meaning
Artificial intelligence (AI)	Broad field: systems that perform tasks associated with intelligent behaviour.
Machine learning (ML)	Sub-field: algorithms whose performance on a task improves from data.
Deep learning (DL)	ML with multi-layer neural networks that learn their own features.
Feature / label	An input measurement / the target answer supplied during supervised training.
Model	A parameterised function fitted to data.
Loss function	A scalar score of prediction error; training minimises its average.
Gradient	The vector of partial derivatives: the direction of steepest ascent.
Backpropagation	Chain-rule algorithm computing the loss gradient for every weight, layer by layer.
Overfitting	Memorising the training set; excellent training scores, poor generalisation.
Embedding	A dense vector encoding of an item's meaning; similarity becomes geometry.
Token	The sub-word unit that language models read and write.
Parameter	A learnable number inside a model; LLMs have billions.
Fine-tuning	Adapting a pretrained model to a new task with a small dataset.
Hallucination	Fluent model output that is not grounded in fact.

— End of document —